

ESTRUTURAS DE DADOS EM PYTHON

1. Introdução às Estruturas de Dados

Uma **estrutura de dados** é uma forma de organizar e armazenar informações na memória para que elas possam ser utilizadas de maneira eficiente.

Em Python, algumas das principais estruturas de dados são:

- Listas (`list`)
- Tuplas (`tuple`)
- Dicionários (`dict`)
- Conjuntos (`set`)
- Strings (`str`)

Neste material, vamos trabalhar principalmente com:

1. Listas;
2. Tuplas;
3. Dicionários;
4. Manipulação de arquivos;
5. Índices;
6. Slicing;
7. Percorrimento de estruturas;
8. Métodos de estruturas de dados.

2. Listas

A **lista** é uma estrutura de dados utilizada para armazenar vários valores em uma única variável.

Uma lista pode conter números, textos, valores booleanos ou até mesmo outras estruturas.

Exemplo

```
numeros = [10, 20, 30, 40, 50]

print(numeros)
```

Saída:

```
[10, 20, 30, 40, 50]
```

Uma lista é:

- ordenada;
- mutável;
- indexada;
- permite valores duplicados;
- pode armazenar diferentes tipos de dados.

Lista com diferentes tipos

```
dados = [10, 5.9, "Python", True]

print(dados)
```

3. Índices

Os elementos de uma lista possuem uma posição chamada **índice**.

Em Python, o primeiro índice é `0`.

Considere:

```
frutas = ["maçã", "banana", "laranja", "uva"]
```

Posição	Índice	Valor
1ª	0	maçã
2ª	1	banana
3ª	2	laranja
4ª	3	uva

Para acessar um elemento:

```
print(frutas[0])
```

Saída:

```
maçã
```

Para acessar o terceiro elemento:

```
print(frutas[2])
```

Saída:

```
laranja
```

Índices negativos

Python também permite acessar elementos começando pelo final.

```
frutas = ["maçã", "banana", "laranja", "uva"]
```

Índice	Valor
-1	uva
-2	laranja
-3	banana
-4	maçã

Exemplo:

```
print(frutas[-1])
```

Saída:

```
uva
```

4. Criando uma lista

Podemos criar uma lista diretamente:

```
numeros = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]  
  
print(numeros)
```

Também podemos criar uma lista usando `range()`:

```
numeros = list(range(10))  
  
print(numeros)
```

Resultado:

```
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

5. Adicionando elementos — `append()`

O método `append()` adiciona um elemento **ao final da lista**.

```
lista = [1, 2, 3]  
  
lista.append(4)  
  
print(lista)
```

Resultado:

```
[1, 2, 3, 4]
```

Outro exemplo:

```
lista.append(10)  
lista.append(20)  
  
print(lista)
```

6. Inserindo elementos — `insert()`

O método `insert()` permite inserir um elemento em determinada posição.

Sintaxe:

```
lista.insert(indice, valor)
```

Exemplo:

```
lista = [10, 20, 30, 40]

lista.insert(2, 99)

print(lista)
```

Resultado:

```
[10, 20, 99, 30, 40]
```

Observe que o valor `99` foi inserido no índice `2`.

7. Removendo elementos

7.1 `remove()`

Remove um elemento pelo seu **valor**.

```
lista = [10, 20, 30, 40]

lista.remove(30)

print(lista)
```

Resultado:

```
[10, 20, 40]
```

O `remove()` remove a primeira ocorrência encontrada.

7.2 `pop()`

O método `pop()` remove um elemento utilizando seu índice.

```
lista = [10, 20, 30, 40]

lista.pop(1)

print(lista)
```

Resultado:

```
[10, 30, 40]
```

Também podemos utilizar:

```
lista.pop()
```

Nesse caso, o último elemento será removido.

8. Contando elementos — `count()`

O método `count()` informa quantas vezes determinado valor aparece na lista.

```
lista = [1, 2, 2, 3, 2, 4]

print(lista.count(2))
```

Resultado:

```
3
```

9. Localizando elementos — `index()`

O método `index()` retorna o índice da primeira ocorrência de determinado valor.

```
lista = [10, 20, 30, 40]

print(lista.index(30))
```

Resultado:

```
2
```

10. Alterando elementos

As listas são **mutáveis**. Isso significa que podemos alterar seus elementos.

```
lista = [10, 20, 30, 40]

lista[2] = 99

print(lista)
```

Resultado:

```
[10, 20, 99, 40]
```

11. Slicing

Slicing é a técnica utilizada para obter uma parte de uma lista.

A sintaxe é:

```
lista[inicio:fim:passo]
```

Considere:

```
lista = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

Primeiros três elementos

```
print(lista[:3])
```

Resultado:

```
[0, 1, 2]
```

Elementos do índice 2 até antes do índice 7

```
print(lista[2:7])
```

Resultado:

```
[2, 3, 4, 5, 6]
```

O índice final não é incluído.

De dois em dois

```
print(lista[::2])
```

Resultado:

```
[0, 2, 4, 6, 8]
```

De três em três

```
print(lista[::3])
```

Resultado:

```
[0, 3, 6, 9]
```

Últimos três elementos

```
print(lista[-3:])
```

Resultado:

```
[7, 8, 9]
```

Todos menos os quatro últimos

```
print(lista[:-4])
```

Resultado:

```
[0, 1, 2, 3, 4, 5]
```

12. Invertendo uma lista

Existem duas formas comuns.

Utilizando `reverse()`

```
lista = [1, 2, 3, 4, 5]

lista.reverse()

print(lista)
```

Resultado:

```
[5, 4, 3, 2, 1]
```

O `reverse()` altera a própria lista.

Utilizando slicing

```
lista = [1, 2, 3, 4, 5]

nova_lista = lista[::-1]

print(nova_lista)
```

Nesse caso, uma nova sequência é obtida.

13. Ordenando listas

O método `sort()` organiza os elementos da lista.

```
lista = [5, 2, 8, 1, 3]

lista.sort()

print(lista)
```

Resultado:

```
[1, 2, 3, 5, 8]
```

Ordem decrescente

```
lista.sort(reverse=True)

print(lista)
```

Resultado:

```
[8, 5, 3, 2, 1]
```

14. Percorrendo uma lista

Podemos utilizar o `for`:

```
frutas = ["maçã", "banana", "laranja"]

for fruta in frutas:
    print(fruta)
```

Saída:

```
maçã
banana
laranja
```

Também podemos acessar os índices:

```
for i in range(len(frutas)):
    print(i, frutas[i])
```

15. Tuplas

Uma **tupla** é semelhante a uma lista, porém possui uma característica fundamental:

A tupla é imutável.

Criamos uma tupla utilizando parênteses:

```
numeros = (0, 1, 2, 3, 4, 5)

print(numeros)
```

Podemos acessar seus elementos:

```
print(numeros[2])
```

Resultado:

```
2
```

16. Lista × Tupla

Característica	Lista	Tupla
Símbolo	[]	()
Ordenada	Sim	Sim
Indexada	Sim	Sim
Mutável	Sim	Não
Permite duplicados	Sim	Sim
append()	Sim	Não
remove()	Sim	Não

Exemplo de alteração em lista

```
lista = [10, 20, 30]

lista[1] = 99

print(lista)
```

Funciona.

Tentativa de alteração em tupla

```
tupla = (10, 20, 30)

tupla[1] = 99
```

Resultado:

TypeError

A tupla não permite alteração direta de seus elementos.

17. Método `index()` em tuplas

Podemos descobrir a posição de um elemento:

```
tupla = (10, 20, 30, 40, 50)

print(tupla.index(40))
```

Resultado:

3

18. Dicionários

Um **dicionário** armazena informações no formato:

chave → valor

Exemplo:

```
aluno = {
    "nome": "Ana",
    "idade": 20,
    "curso": "Computação"
}
```

Nesse exemplo:

- `"nome"` é uma chave;
 - `"Ana"` é o valor;
 - `"idade"` é uma chave;
 - `20` é o valor correspondente.
-

19. Acessando valores de um dicionário

```
print(aluno["nome"])
```

Resultado:

```
Ana
```

Outro exemplo:

```
print(aluno["curso"])
```

Resultado:

```
Computação
```

20. Adicionando uma entrada

Podemos adicionar uma nova chave:

```
aluno["periodo"] = 3  
  
print(aluno)
```

21. Alterando valores

```
aluno["idade"] = 21  
  
print(aluno)
```

O valor associado à chave `"idade"` foi alterado.

22. Principais métodos dos dicionários

`keys()`

Retorna as chaves.

```
print(aluno.keys())
```

`values()`

Retorna os valores.

```
print(aluno.values())
```

`items()`

Retorna pares chave/valor.

```
print(aluno.items())
```

23. Percorrendo um dicionário

Podemos percorrer somente as chaves:

```
for chave in aluno:  
    print(chave)
```

Podemos percorrer os valores:

```
for valor in aluno.values():  
    print(valor)
```

Ou podemos percorrer chave e valor simultaneamente:

```
for chave, valor in aluno.items():  
    print(f"{chave} tem como valor {valor}")
```

24. Estrutura de arquivos

Além das estruturas de dados em memória, Python permite trabalhar com arquivos.

Um arquivo pode ser utilizado para **armazenar dados permanentemente**.

A função principal é:

```
open()
```

Exemplo:

```
arquivo = open("dados.txt", "w")
```

Entretanto, a forma recomendada é utilizar `with`:

```
with open("dados.txt", "w") as arquivo:  
    arquivo.write("Python")
```

O `with` garante o fechamento adequado do arquivo após a utilização.

25. Modos de abertura de arquivos

Modo	Significado	Utilização
"r"	Read	Leitura
"w"	Write	Escrita/substituição
"a"	Append	Adicionar ao final

"w"

```
with open("dados.txt", "w") as arquivo:  
    arquivo.write("Olá Python!")
```

Se o arquivo já possuir conteúdo, ele será substituído.

"a"

```
with open("dados.txt", "a") as arquivo:  
    arquivo.write("\nNovo conteúdo")
```

O novo conteúdo será acrescentado ao final.

26. Escrevendo números em um arquivo

```
with open("numeros.txt", "w") as arquivo:
    for numero in range(1, 11):
        arquivo.write(f"{numero}\n")
```

O `\n` representa uma **quebra de linha**.

O arquivo ficará:

```
1
2
3
4
5
6
7
8
9
10
```

27. Lendo um arquivo

Podemos utilizar `read()`:

```
with open("numeros.txt", "r") as arquivo:
    conteudo = arquivo.read()

print(conteudo)
```

28. Lendo linha por linha

Outra possibilidade:

```
with open("numeros.txt", "r") as arquivo:
    for linha in arquivo:
        print(linha.strip())
```

O `strip()` remove espaços e caracteres de quebra de linha das extremidades da string.

29. Arquivos e estruturas de dados

Podemos combinar arquivos com listas.

Exemplo:

```
with open("numeros.txt", "r") as arquivo:
    linhas = arquivo.readlines()

print(linhas)
```

O resultado será uma lista contendo as linhas do arquivo.

Podemos transformar os valores em inteiros:

```
with open("numeros.txt", "r") as arquivo:
    numeros = [int(linha.strip()) for linha in arquivo]

print(numeros)
```

Resultado:

```
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

Esse exemplo mostra uma aplicação prática de **listas + arquivos + conversão de tipos**.

30. Principais estruturas estudadas

Lista

```
lista = [10, 20, 30]
```

Utilizada quando precisamos de uma coleção **mutável e ordenada**.

Tupla

```
tupla = (10, 20, 30)
```

Utilizada quando queremos uma coleção **ordenada e imutável**.

Dicionário

```
aluno = {  
    "nome": "Ana",  
    "idade": 20  
}
```

Utilizado quando precisamos relacionar **chaves e valores**.

31. Comparação geral

Estrutura	Sintaxe	Mutável	Acesso
Lista	[]	Sim	Índice
Tupla	()	Não	Índice
Dicionário	{chave: valor}	Sim	Chave
String	"texto"	Não	Índice

32. Atividade prática

Desafio 1 — Lista de alunos

Crie uma lista contendo cinco nomes.

Faça:

1. Imprima a lista;
2. Adicione um sexto aluno;
3. Remova um aluno;
4. Altere o terceiro nome;
5. Imprima os três primeiros nomes;
6. Imprima os dois últimos nomes;
7. Inverta a lista;
8. Ordene a lista em ordem alfabética.

Desafio 2 — Notas

Crie uma lista:

```
notas = [7.5, 8.0, 6.5, 9.0, 5.5]
```

Faça:

1. Calcule a maior nota;
2. Calcule a menor nota;
3. Calcule a quantidade de notas;
4. Calcule a soma das notas;
5. Calcule a média;
6. Ordene as notas;
7. Imprima as três maiores notas.

Dica:

```
max()  
min()  
len()  
sum()
```

33. Desafio 3 — Cadastro com dicionário

Crie um dicionário representando um aluno:

```
aluno = {  
    "nome": "Maria",  
    "idade": 22,  
    "curso": "ADS",  
    "periodo": 4,  
    "nota": 8.5  
}
```

Faça:

1. Imprima o nome;
2. Imprima o curso;
3. Altere a idade;
4. Adicione a chave "cidade";
5. Percorra todas as chaves;
6. Percorra todos os valores;
7. Percorra chave e valor;
8. Imprima o dicionário completo.

34. Desafio 4 — Lista de dicionários

Agora vamos trabalhar com uma estrutura mais próxima de aplicações reais.

Crie:

```
alunos = [  
    {"nome": "Ana", "nota": 8.5},  
    {"nome": "Carlos", "nota": 7.0},  
    {"nome": "Maria", "nota": 9.2},  
    {"nome": "João", "nota": 6.5}  
]
```

Percorra a estrutura:

```
for aluno in alunos:  
    print(aluno["nome"], aluno["nota"])
```

Resultado:

```
Ana 8.5  
Carlos 7.0  
Maria 9.2  
João 6.5
```

Essa estrutura é muito importante porque representa um padrão comum em sistemas de software:

```
Lista  
├── Dicionário  
│   ├── nome  
│   └── nota  
├── Dicionário  
│   ├── nome  
│   └── nota  
└── Dicionário  
    ├── nome  
    └── nota
```

Ela pode representar dados vindos de APIs, bancos de dados, arquivos JSON e sistemas web.

35. Boas práticas

Ao trabalhar com estruturas de dados em Python:

1. Escolha a estrutura adequada

Use:

- `list` → coleção ordenada e mutável;
- `tuple` → coleção ordenada e imutável;
- `dict` → dados organizados por chave;
- `set` → coleção de valores únicos.

2. Utilize nomes significativos

Evite:

```
x = [...]
```

Prefira:

```
alunos = [...]
```

3. Utilize `with` para arquivos

Prefira:

```
with open("dados.txt", "r") as arquivo:  
    ...
```

em vez de abrir e esquecer de fechar o arquivo.

4. Observe os índices

Lembre-se:

Em Python, a contagem dos índices começa em `0`.

Portanto:

```
1ª posição → índice 0  
2ª posição → índice 1  
3ª posição → índice 2
```

36. Resumo da aula

Nesta aula estudamos:

- Conceito de estrutura de dados;
- Listas;
- Índices;
- Índices negativos;
- `append()`;
- `insert()`;
- `remove()`;
- `pop()`;
- `count()`;
- `index()`;
- `reverse()`;
- `sort()`;
- Slicing;
- Tuplas;
- Imutabilidade;
- Dicionários;
- Chaves e valores;
- `keys()`;
- `values()`;
- `items()`;
- Percorrimento com `for`;
- Manipulação de arquivos;
- Modos `r`, `w` e `a`;
- `read()`;
- `write()`;
- `readlines()`;
- `strip()`;
- Combinação de estruturas de dados.

Conceito-chave

Estruturas de dados são fundamentais para organizar, manipular e processar informações dentro de um programa.

A escolha adequada da estrutura influencia diretamente a **organização, legibilidade, manutenção e eficiência** do código.